SOFTWARE REQUIREMENTS SPECIFICATION

DataNexus

Automated Data Quality & Observability Framework

Document Type	Software Requirements Specification (SRS)
Project	DEPI Graduation Project 2026
Date	February 2026

1. Introduction

This SRS defines the functional and non-functional requirements for DataNexus — an automated data quality and observability framework. It acts as the baseline agreement between stakeholders and the development team, governing design, implementation, and acceptance.

1.1 Scope

In Scope	Out of Scope
Automated data profiling & statistical analysis	Data transformation logic (dbt / Airflow)
YAML-driven configurable validation rule engine	Data warehousing or storage infrastructure
Real-time quality monitoring & multi-channel alerts	BI / visualization tools (Tableau, Power BI)
Historical trend tracking & schema-drift detection	Data governance and cataloging
REST API, CLI tool, and Streamlit dashboard	Data lineage tracking [planned for v2.0]
Apache Airflow scheduling & GitHub Actions gate	—

1.2 Stakeholder Register

ID	Stakeholder	Role	Primary Need	Priority
SH-01	Data Engineer	Primary user	Reduce debugging time by 70%	High
SH-02	Data Team Lead	Oversight	Real-time quality dashboard per pipeline	High
SH-03	Business Analyst / CFO	Consumer	Reliable reports; zero bad-data incidents	Medium

ID	Stakeholder	Role	Primary Need	Priority
SH-04	DevOps / CI Engineer	Infrastructure owner	Seamless GitHub Actions integration	Medium
SH-05	Senior Architect	Technical governance	Modular, scalable, production-ready code	Medium

1.3 Definitions

Term	Definition
Quality Score	Weighted percentage (0–100) representing overall data health for a single run
Severity	Impact classification of a failed check: Low / Medium / High / Critical
Schema Drift	Unplanned changes to column names, data types, or table structure
Validation Run	One complete execution of all checks defined in a validation config
Great Expectations	Open-source Python library providing an extended suite of data validators
DAG	Directed Acyclic Graph — the unit of work in Apache Airflow
Quality Gate	CI/CD step that blocks a merge if Quality Score falls below a threshold

2. Functional Requirements

ID	Title	Priority	Stakeholder
FR-01	Automated Data Profiling	Mandatory	SH-01, SH-02
FR-02	Configurable Validation Engine	Mandatory	SH-01
FR-03	Real-Time Monitoring & Alerting	Mandatory	SH-01, SH-02, SH-03
FR-04	Historical Quality Tracking	Mandatory	SH-02, SH-03
FR-05	CI/CD Integration	Mandatory	SH-04
FR-06	Multi-Interface Access	Mandatory	SH-01, SH-04

FR-01 — Automated Data Profiling

The system shall automatically analyze any dataset and produce a comprehensive statistical profile without manual configuration. Supported sources: CSV, PostgreSQL, MySQL, MongoDB, JSON.

- Compute per-column statistics: count, null %, distinct count, min, max, mean, median, stddev, and percentiles (P25, P50, P75, P95, P99).
- Auto-detect common data patterns: email addresses, phone numbers, dates, and identifier formats.
- Identify statistical outliers using configurable thresholds.
- Detect schema drift by comparing the current profile against stored historical snapshots.
- Profiling frequency configurable: on-demand, daily, or per pipeline run.

FR-02 — Configurable Validation Engine

The system shall execute data quality checks defined entirely in YAML/JSON configuration files — no code changes required to add or modify rules.

Category	Supported Check Types	Example
Completeness	not_null, completeness	email non-null >= 95%
Accuracy	regex, range, in_set	age in [18, 120]; email matches RFC pattern

Category	Supported Check Types	Example
Uniqueness	unique	customer_id must be 100% unique
Consistency	foreign_key, in_set, custom_sql	every order_id references a valid customer
Timeliness	freshness	updated_at within the last 24 hours

- Each check requires: name, category, check_type, target column, threshold (0.0–1.0), and severity (Low / Medium / High / Critical).
- Config schema is validated at load time; violations return descriptive error messages pointing to the exact field.
- Integrates with Great Expectations 0.18+ for advanced, battle-tested validators.

FR-03 — Real-Time Monitoring & Alerting

The system shall calculate a Quality Score per validation run and dispatch alerts through configured channels when thresholds are breached.

- Quality Score (0–100) is severity-weighted: Critical=1.0, High=0.75, Medium=0.5, Low=0.25.
- Alert channels: Email (SMTP), Slack Webhook, PagerDuty, and custom webhooks.
- Each alert includes: dataset name, failed check, expected vs. actual value, severity, failed row count, and a direct dashboard link.
- Duplicate-alert suppression within a configurable cooldown window.
- Auto-escalation policy re-alerts if a Critical issue remains unacknowledged after a configurable timeout.

FR-04 — Historical Quality Tracking

The system shall persist all validation run data and expose trend analysis for quality monitoring over time.

- Store per-check results: expected/actual values, threshold, status, and a sample of up to 10 failed rows.
- Expose time-series quality score trends per dataset through both the dashboard and the API.
- Record pipeline performance metrics: execution time, record count, and SLA compliance flag.
- Default retention: runs 90 days, profiles 1 year, pipeline metrics 2 years (configurable).

FR-05 — CI/CD Integration

The system shall integrate natively with GitHub Actions and Apache Airflow to enforce quality gates automatically.

- CLI returns exit code 0 on pass and exit code 1 on failure, enabling direct shell-based CI/CD integration.
- A configurable Quality Gate threshold blocks pull request merges when the score falls below the defined minimum.
- Supports scheduled DAG-based execution via Apache Airflow 2.7+.
- Ready-to-use GitHub Actions workflow templates provided for zero-configuration onboarding.

FR-06 — Multi-Interface Access

The system shall be accessible through three distinct interfaces to serve different user personas.

Interface	Technology	Key Capabilities
Web Dashboard	Streamlit 1.28+	Quality score gauges, trend charts, alert management, config editor
CLI Tool	Click + Rich	Run validations, view history, generate config templates, colored output
REST API	Flask 3.0+ or FastAPI 0.110+	CRUD for configs/datasets, trigger runs, query results, receive webhooks

3. Non-Functional Requirements

3.1 Performance

ID	Requirement	Target	Measurement Method
NFR-P01	Validation throughput	< 5 sec for 1,000,000 rows (4-core / 16 GB RAM)	pytest performance suite
NFR-P02	API response time (P95)	< 200 ms	Locust load test
NFR-P03	Dashboard load time	< 3 seconds	Browser timing API
NFR-P04	DB query time (P95)	< 100 ms	PostgreSQL slow-query log

3.2 Reliability

ID	Requirement	Target
NFR-R01	System uptime	>= 99.0%
NFR-R02	Fault tolerance	Partial check failure must not abort the run; all completed results must be stored
NFR-R03	DB connection resilience	Exponential back-off retry, max 3 attempts before transitioning run to error state
NFR-R04	Result persistence guarantee	Run metadata written to DB before any alert is dispatched

3.3 Scalability

ID	Requirement	Target
NFR-SC01	Horizontal scaling: API and validation workers must support multiple stateless container replicas behind a load balancer	>= 3 replicas with no shared in-process state
NFR-SC02	Concurrent validation runs: handle >= 10 simultaneous pipeline executions without result cross-contamination	>= 10 concurrent runs with no data corruption
NFR-SC03	Dataset size growth: profiling and validation engine must sustain throughput from 10K to 10M rows	Linear time complexity; no > 2x throughput degradation per 10x row increase

3.4 Security

ID	Requirement
NFR-S01	All credentials and API keys must be injected via environment variables — never stored in source code or logs
NFR-S02	REST API endpoints must be protected by JWT authentication
NFR-S03	Docker containers must run as non-root users
NFR-S04	Config files support ${ENV_VAR} substitution for webhook URLs and sensitive values

3.5 Maintainability & Code Quality

ID	Metric	Target	Tool
NFR-M01	Test coverage	>= 80%	pytest-cov
ID	**Metric**	**Target**	**Tool**

NFR-M02	Linting compliance	0 critical issues	Flake8 + Black
NFR-M03	Type annotation	>= 80%	mypy
NFR-M04	Security scan	0 High / Critical	Bandit + pip-audit

4. Data Model

Entity	Purpose	Key Attributes	Constraints
data_sources	Registered data connections	name, type, connection_string	name UNIQUE; type IN (postgresql, mysql, mongodb, csv, json)
datasets	Tables / files under validation	data_source_id (FK), schema_name, table_name	UNIQUE(data_source_id, schema_name, table_name)
data_profiles	Statistical snapshots	dataset_id (FK), profile_date, profile_json	profile_json NOT NULL
validation_configs	YAML rule definitions	dataset_id (FK), config_yaml, schedule_cron	config_yaml NOT NULL; UNIQUE(dataset_id, name)
validation_runs	Execution records	config_id (FK), status, quality_score	status IN (pass,fail,error,running); quality_score [0,100]
validation_results	Per-check detail	run_id (FK), check_name, status, severity	severity IN (low, medium, high, critical)
alerts	Dispatched notifications	run_id (FK), channel, severity, acknowledged	channel IN (email, slack, pagerduty, webhook)
pipeline_metrics	Performance & SLA tracking	dataset_id (FK), metric_date, sla_met	sla_met BOOLEAN; metric_date NOT NULL

Retention Policy

Data Type	Default Retention
Validation run results	90 days
Data profiles	1 year
Alert history	180 days
Pipeline metrics	2 years

5. Use Cases

UC	Title	Actor	Trigger	Main Success Outcome
UC-01	Manual Validation Run	Data Engineer	CLI command executed	Quality Score displayed; results persisted to DB
UC-02	Scheduled Validation Run	Airflow Scheduler	Cron trigger fires	Run stored; alert dispatched if threshold breached
UC-03	View Quality Dashboard	Engineer / Lead	User opens Streamlit URL	Live score, trend chart, and active alerts visible
UC-04	Receive & Acknowledge Alert	Data Engineer	Validation fails above severity	Slack/Email received; acknowledged via dashboard
UC	**Title**	**Actor**	**Trigger**	**Main Success Outcome**

UC-05	CI/CD Quality Gate	GitHub Actions	Pull request opened	PR blocked or approved based on Quality Score
UC-06	Add New Validation Rule	Data Engineer	Engineer edits YAML config	New check executes on next run with zero code change
UC-07	Investigate Schema Drift	Data Engineer	Schema drift alert received	Column-level diff and historical stats viewed in dashboard

6. Constraints & Assumptions

6.1 Technical Constraints

ID	Constraint	Impact
CON-01	Python 3.10+ required	Incompatible with Python 2 and versions < 3.10
CON-02	PostgreSQL 14+ for metadata storage	JSONB column type unavailable in earlier versions
CON-03	Docker required for containerized deploy	Cannot run on environments without a Docker runtime
CON-04	Apache Airflow 2.7+ for scheduling	DAG syntax differs in earlier Airflow versions
CON-05	Outbound internet for alert delivery	Slack and Email alerts non-functional in air-gapped environments. Mitigation: use custom webhook channel or disable alerting and rely on CLI exit codes + dashboard.

6.2 Assumptions

- Users possess basic YAML knowledge to author validation configurations.
- Source databases are network-accessible from the deployment environment.
- Sufficient PostgreSQL storage is available for validation history and profiles.
- The development team has Docker and Python 3.10+ available on all workstations.

7. Requirements Traceability Matrix

Req ID	Business Problem	Component	Test Module	Use Case
FR-01	Silent failures; no data visibility	Profiler Engine	test_profiler.py	UC-03, UC-07
FR-02	Manual checks; not scalable or reusable	Validator Engine + Config Parser	test_validators.py	UC-01, UC-06
FR-03	3-week mean time to detect data issues	Alert Manager + Notifier	test_alerts.py	UC-04
FR-04	No trend analysis or regression detection	PostgreSQL + Dashboard	test_history.py	UC-03
FR-05	Bad data reaches production undetected	GitHub Actions Workflow	tests.yml	UC-05
FR-06	Accessibility for different personas	Streamlit + Flask + Click	test_api.py, test_cli.py	UC-01–05

8. Acceptance Criteria & Success Metrics

8.1 Milestone Gates

Milestone	Delivery	Acceptance Criteria
M1 – MVP	End of Week 4	5+ validators operational; DB schema live; CLI returns correct exit codes; results persisted to DB
M2 – Usable Product	End of Week 6	REST API and dashboard live with real data; 10+ validators; data profiling functional; Great Expectations integrated
M3 – Production-Ready	End of Week 8	Email + Slack alerts operational; CI/CD pipeline configured; test coverage >= 80%; full Docker Compose stack healthy
M4 – Demo-Ready	End of Week 10	Complete documentation; demo scenarios prepared; video recorded; GitHub repository public and open-sourced

8.2 Key Performance Indicators

KPI	Baseline	Target	Measurement Method
Mean time to detect quality issues	~3 weeks	< 1 hour	Time from data injection to alert received
Data engineer debugging time	60% of hours	< 20%	Time-tracking over a 2-week sprint
Production data incidents	~10 / month	<= 2 / month	Incident log review
Average pipeline quality score	~75%	>= 95%	Dashboard aggregate score
Test coverage	0%	>= 80%	pytest-cov HTML report

8.3 Technology Stack Summary

Component	Technology	Version
Language	Python	3.10+
Orchestration	Apache Airflow	2.7+
Metadata Database	PostgreSQL	14+
Validation Library	Great Expectations	0.18+
Web Framework	Flask 3.0+ or FastAPI 0.110+	3.0+
Dashboard	Streamlit	1.28+
CLI	Click + Rich	8.x
Containerization	Docker + Docker Compose	Latest
CI/CD	GitHub Actions	—
Testing	pytest + pytest-cov	7.4+

9. Error Handling & System Responses

The system shall handle failure conditions gracefully, ensuring no data loss and providing actionable feedback to all user interfaces.

ID	Error Condition	System Response	User Notification
EH-01	Invalid YAML / JSON config schema	Reject config at load time; abort run before any check executes; no partial results written	CLI: descriptive error with exact field path; API: HTTP 422 with validation detail; Dashboard: inline red banner

ID	Error Condition	System Response	User Notification
EH-02	Database connection failure during run	Exponential back-off (NFR-R03); after 3 failures, transition run to "error" state and persist partial results	CLI: exit code 2 with timestamped error log; API: HTTP 503; Alert dispatched via all configured channels
EH-03	Individual check execution failure	Mark check as "error" (not fail); continue remaining checks per NFR-R02; overall run continues	Dashboard: check card shows "Error" badge with exception trace link; excludes errored checks from Quality Score
EH-04	Alert channel delivery failure (Slack/Email)	Retry once after 30s; log delivery failure to DB; attempt fallback channel if configured; run unaffected	Dashboard: alert status shows "Delivery Failed" with retry timestamp; system log captures full exception
EH-05	Unauthorized API request (invalid/missing JWT)	Reject immediately; no data returned; request logged with IP and timestamp for audit	HTTP 401: {"error": "Unauthorized", "hint": "Provide a valid Bearer token"}

10. Risk Register

Likelihood and Impact scored 1 (Low) – 5 (Critical). Score = Likelihood × Impact. Score >= 10 = High Risk.

ID	Risk Description	L	I	Score	Mitigation Strategy
RISK-01	Performance target (NFR-P01) not met for datasets > 1M rows with complex checks (regex + foreign key)	3	4	12	Implement Pandas chunking + vectorised operations from Sprint 1; run benchmark suite weekly; downgrade to sampling mode if threshold not met
RISK-02	Great Expectations 0.18+ API breaking changes before project completion	2	3	6	Pin GE version in requirements.txt; wrap GE calls in an adapter layer; monitor GE changelog weekly
RISK-03	Team scope creep — adding features beyond SRS during sprint (data lineage, BI connectors)	4	3	12	Use this SRS as change-control baseline; new features require formal approval; out-of-scope items logged in backlog for v2.0
RISK-04	JWT secret or DB credentials accidentally committed to public GitHub repository	2	5	10	Enforce .gitignore for .env files; enable GitHub Secret Scanning; use GitHub Actions secrets for CI; mandate NFR-S01 pre-commit hook